Programming II

Nested and Anonymous Classes

Michele Pasqua, PhD

UNIVERSITÀ
di VERONA
Dipartimento
di INFORMATICA

A.Y. 2025/2026

Nested Classes

Nested class

A (static) class can be defined inside another class *// called nested*

```
package pkg;
class Outer {
  static class Nested {
    ...
  }
}
```

Similar to regular classes

- Same visibility rules
- Full name includes the outer class: `pkg.Outer.Nested`

Nested class (cont'd)

A (static) class can be defined inside another class // *called nested*

They are static member of the enclosing class

- Have access to other `static` members of the enclosing class // *even private*
- Can be used independently from the enclosing class

```
// create an object for the nested class                    1
Outer.Nested nestedObject = new Outer.Nested();             2
```

Inner class

A (non-static) class can be defined inside another class *// called inner*

```
package pkg;
class Outer {
  class Inner {
    ...
  }
}
```

Similar to regular classes

- Same visibility rules
- Full name includes the outer class: `pkg.Outer.Inner`

Inner class (cont'd)

A (non-static) class can be defined inside another class // *called inner*

An inner class instance cannot exists outside an outer class instance

- Have access to other members of the enclosing class // *even private*
- Cannot be used independently from the enclosing class

```
// create an object for the nested class                    1
Outer outerObject = new Outer();                            2
Outer.Inner innerObject = Outer.new Inner();                3
```

Nested vs inner classes

```
class Outer {
  private static int I = 5;
  private int i;

  static class Nested { // does not have access to instance member i
    static void print() { System.out.println(I); }
  }
  class Inner { // has access to the instance member i
    void print() { System.out.println(i); }
  }

  public void outerMethod() {
    Nested.print();
    Inner innerObject = new Inner();
    innerObject.print();
  }
}
```

Local class

A non-static class can be defined inside another class method // *called local*

- Instantiation and variable access is limited by the enclosing method

```
public void outerMethod() {
  int j = 1;

  class Local { // no access modifier
    int addOne() { return j + 1; }
  }

  Local localObject = new Local();
  System.out.println(localObject.addOne()); // the output is '2'
}
```

Local class (cont'd)

A non-static class can be defined inside another class method // called local

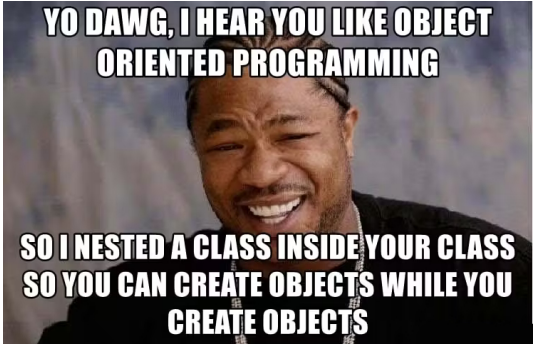
- Can only access `final` variables // some kind of closure

```
public void outerMethod() {
  final int j = 1;

  class Local { // no access modifier
    int addOne() { return j + 1; } // j++ not allowed
  }

  Local localObject = new Local();
  System.out.println(localObject.addOne()); // the output is '2'
}
```

Objects inside objects



Why class nesting?

Better code organization

- Sometimes it is better to put a class into the namespace of another class

Easier visibility policy

- Nested classes have special access to variables from the enclosing class

Less complex codebase

- No need create a specific file for classes with limited scope

Why class nesting? (cont'd)

```
class ClassRoom implements Iterable {
  private Student[] students;

  ClassRoom(Student[] students)
  { this.students = students; }
  @Override
  public Iterator iterator()
  { return new CRIterator(); }
}
```

```
class CRIterator implements Iterator {
  private Student[] arr; private int next = 0;

  CRIterator(Student[] arr) { this.arr = arr; }

  @Override
  public boolean hasNext()
  { return next < arr.length; }
  @Override
  public Object next()
  { return new Student(arr[next++]); }
}
```

```
class ClassRoom implements Iterable {
  private Student[] students;

  ClassRoom(Student[] students) {
    this.students = students;
  }
  @Override
  public Iterator iterator() {
    return new CRIterator(students);
  }

  class CRIterator implements Iterator {
    private int next = 0;

    @Override
    public boolean hasNext() {
      return next < students.length;
    }
    @Override
    public Object next() {
      return new Student(students[next++]);
    }
  }
}
```

Anonymous Classes

Anonymous class

A local class having no name

- Typically, local classes are instantiated and used only once
- Only possible with inheritance

Extend a class

`new SomeParentClass(` *constructor arguments* `) {` *method declarations* `}`

```
new Person("Sam") {
  @Override
  public String toString() { return "It's Sam!"; }
}
```

Anonymous class (cont'd)

A local class having no name

- Typically, local classes are instantiated and used only once
- Only possible with inheritance

Implement an interface

`new SomeInterface() { `*method declarations*` }`

```
new Runnable() {
  @Override
  public void run() { ... }
}
```

Anonymous class usage

Anonymous classes are expressions

- They must be part of a statement

Left-hand side of an assignment

```
Person sam = new Person("Sam") {
  @Override
  public String toString() { return "It's␣Sam!"; }
};
System.out.println(sam.toString());
```

Method actual parameter

```
List<Runnable> tasks = new ArrayList<>();
tasks.add(new Runnable() {
  @Override
  public void run() { ... }
});
```

⚠ We cannot specify a constructor for anonymous classes

Lambdas

Simplified anonymous classes, implementing a single method interface

```
public interface SingleInterface { int singleMethod(int param); }
```

```
SingleInterface singleInc = new SingleInterface() { // anonymous class
  @Override
  public int singleMethod(int param) { return param + 1; }
};
SingleInterface singleDec = param -> param - 1; // with a lambda

System.out.println(singleInc.singleMethod(3)); // the output is '4'
System.out.println(singleDec.singleMethod(3)); // the output is '2'
```

Lambdas syntax

parameters -> *body*

Where *parameters* can be

- Zero: `()`
- One: `x`
- Two or more: `(x, y)`

And *body* can be

- An expression: `x + y`
- A code block: `{ return x + y; }`

```
// types are automatically inferred
```

Why lambdas?

Make the code more concise and readable

- Focus on the functionality rather than implementation
- Add a flavor of functional programming

They still have limitations wrt anonymous classes

- No multiple methods can be defined
- Inherently stateless, no local variables can be declared

Varargs

It is possible to pass a variable number of arguments to a method

`methodName(`*type*`... args)`

```
static int min(int... values) { // values is translated into an array
  int res = Integer.MAX_VALUE;  // with the correct size
  for (int v : values)
    if (v < res) res = v;
  return res;
}

public static void main(String[] args) {
  System.out.println(min(1, 0, 5));       // the output is '0'
  System.out.println(min(3, 7, 2, 6, 9)); // the output is '2'
}
```

Varargs (cont'd)

Reduce boilerplate code, can be used in lambdas and anonymous classes

```
public interface SingleInterface { int singleMethod(int... params); }
```

```
SingleInterface singleInc = new SingleInterface() { // anonymous class
  @Override
  public int singleMethod(int... params) { return params[0] + 1; }
};
SingleInterface singleDec = params -> params[0] - 1; // with a lambda

System.out.println(singleInc.singleMethod(3, 4, 5)); // the output is '4'
System.out.println(singleDec.singleMethod(3, 2));    // the output is '2'
```

Q + A